Simulace paprskové optiky s grafickým rozhraním

Dokumentace k maturitní práci

Daniel Matouš
Gymnázium, Praha 4, Písnická 760
16. dubna 2026

Abstrakt

Tato dokumentace popisuje grafické rozhraní pro program na simulaci paprskové optiky ve 2D prostoru. Cílem bylo vytvořit desktopovou aplikaci, jejíž GUI má všechny náležitosti očekávané od moderního rozhraní. Dal jsem si za cíl nepoužít jiné knihovny než ty, které jsou součástí základního Pythonu 3.14; všechny systémy projektu jsem se chtěl naučit vymyslet a implementovat sám.

Pro projekt jsem vytvořil základ optické simulace, která v aktuálním stavu podporuje bodové zdroje libovolného počtu paprsků, odrážení od zrcadel a překážky blokující paprsky. Podporuje přidávání dalších prvků odvozením od obecných tříd.

Zásadním bodem cíle byl vlastní systém vybírání a přesouvání simulačních prvků pomocí myši metodou "Drag n' drop".

GUI bylo opatřeno i méně častými widgety a ve svém aktuálním stavu vyladěno, ale není dost vybavené funkcemi, aby dovolilo uživateli využít všech možností simulace. Simulace sice poskytuje funkční základ, na kterém se dá expandovat, ale postrádá schopnosti, které jsem chtěl přidat.

V průběhu tvorby jsem totiž zjistil, že bude komplikovaným výkonem zprovoznit i jen úplný základ. To se mi nakonec povedlo a v této dokumentaci vás provedu jeho tvorbou, fungováním a dalším využitím.

Obsah

1 Úvod **1**

2 Programátorská dokumentace **3**
2.1 Principy fungování 3
2.2 Drag 'n drop 3
2.3 Architektura aplikace 4
2.4 Formátování kódu 5
2.5 Simulace 6
2.6 Popis důležitých částí kódu 6
2.7 Debugovací funkce 11
2.8 Použité technologie a nástroje 12

3 Uživatelská dokumentace **13**
3.1 Instalace a spuštění 13
3.2 Popis GUI 13
3.3 Použití 15
3.4 Režimy výběru myší 15
3.5 Řešení problémů 15

4 Závěr **16**

1 Úvod

Jakmile jsem zjistil, že předmětem závěrečného ročníku bude tvorba maturitního projektu, tak jsem věděl, že chci tvořit nějaké grafické rozhraní. Tím jsem si byl od začátku jistý také proto, že v budoucnu bych chtěl pracovat právě na vývoji nástrojů s GUI. Dále mi bylo jasné, že budu chtít tvořit čistě softwarový projekt bez hardwarové součásti, protože s prací s hardwarem nemám žádné dobré zkušenosti.

Výběr platformy pro mě byl také jasný. Mám zkušenosti s Pythonem a jeho knihovnou Tkinter, které bych si rád prohloubil.

Jako obsah své práce jsem si vymyslel vykreslování paprskové optiky, protože to je záležitost, která se bez dobrého grafického rozhraní neobejde, a proto jsem počítal s tím, že ho nechá vyniknout. Při sepisování požadavků na svou simulaci jsem počítal s využitím jako program, který by studenti mohli využít při studiu optiky pro prohloubení intuice nebo profesoři pro vytváření obrázků optických soustav bez nutnosti používat online nebo placené nástroje. V plánu jsem tedy měl plno simulačních prvků, které by zajistily, že pomocí nich půjde vytvořit skoro cokoliv, co by zamýšlený uživatel mohl chtít.

Udělat tak všestranný program, aby se stal pro někoho první volbou pro práci s optikou, se mi upřímně nepovedlo.

Existují již nástroje, které dělají to, co můj, a k tomu ještě mnohem víc. Na jeden takový nástroj, PhyDemo Ray Optics Simulator, mě upozornil můj profesor hned, jak jsem tento nápad na projekt navrhl. Mým cílem zde ale není vyvinout program schopný konkurovat schopnostmi projektům od mnohem zkušenějších vývojářů (nebo týmů vývojářů), než jsem já, nýbrž ukázat na funkčním příkladu, co umím vytvořit pomocí své oblíbené GUI knihovny.

Cílem tohoto tématu bylo naučit se co nejvíc o tvorbě dobrého uživatelského rozhraní a konkrétně o tvorbě pomocí Tkinteru. V tomto ohledu jsem uspěl, protože kromě základních funkcí úspěšně využívám pokročilejší schopnosti knihovny, včetně používání značek (tagů) pro identifikaci objektů na obrazovkách a dále včetně událostmi řízeného programování a využití rozšiřujícího modulu Tkinter.ttk.

Naučil jsem se hodně jak o principech tvorby grafického rozhraní a mnoha často používaných systémech (práce s mřížkou, resizování, drag 'n drop, událostmi řízené programování), tak o Tkinteru samotném (tvorba menu lišty, systém událostí v Tkinteru, Canvas tagy). Dokážu nyní v této knihovně efektivně pracovat a chápu její součásti. Důvěrně jsem se seznámil i s widgety z jejího rozšiřujícího modulu Tkinter.ttk (ttk = Themed TKinter).

V této dokumentaci najdete informace k fungování projektu a k tomu, jak jej používat. Nejdříve naleznete programátorskou dokumentaci, pak následuje uživatelská dokumentace. V programátorské části je vysvětlena struktura mého projektu a jsou tam popsány nejdůležitější jeho součásti. Popsáno je nejprve také prostředí, ve kterém jsem pracoval a rovněž použité knihovny. Uživatelská dokumentace se sestává z návodu na instalaci a spuštění programu a z návodu k použití.

2 Programátorská dokumentace

V programátorské dokumentaci vás seznámím nejdříve se základními principy, na kterých můj projekt funguje a potom vše rozeberu více do hloubky v částech postupně o souborové struktuře, stavbě grafického rozhraní, matematické simulaci, použitých vývojových nástrojích a nakonec popíšu, jak se v kódu projektu orientovat a vše potřebné pro ty, kdo by v něm chtěli dělat změny.

2.1 Principy fungování

Všechny části projektu ve vysoké míře využívají OOP. Většina souborů projektu poskytuje třídy, se kterými pak `main.py` pracuje. Vstup je zajišťován za pomoci Tkinteru prostřednictvím funkce "bind()"a vestavěného systému událostí (eventů). Využito je tedy událostmi řízené programování, zatímco hlavní proces je po inicializaci GUI ponechán v nekonečném cyklu Tk.mainloop().

2.2 Drag 'n drop

Zásadním bodem cíle byl vlastní systém vybírání a přesouvání simulačních prvků pomocí myši metodou "Drag n' drop", které dovoluje rychle i přesně přemisťovat prvky a je nepostradatelnou součástí každého moderního GUI ovládajícího program, jako je můj.

Drag n' drop je vytvořen za pomocí systému eventů přítomném v Tkinteru. Když uživatel stiskne tlačítko, je zavolána funkce get_mouse_selected, která dostane jako argument událost, ze které získá souřadnice, na kterých událost vznikla, tedy souřadnice kliknutí. Zeptá se Tkinter Canvas widgetu, jaké objekty se na tomto místě nacházejí. Jako odpověď dostane interní ID objektů na Canvasu. Pro každé toto ID se poté zeptá canvasu, jaký je typ toho objektu. Všechny objekty, které jsou čáry, jsou ignorovány. Tímto získá

seznam překážek, které uživatel právě chytil. Uloží je do seznamu. Pokud je aktuální režim vybírání "Vrchní překážka", vybere a uloží jen tu nejvíc vrchní z překážek, co najde pod kurzorem.

Pohnutí myší s podrženým levým tlačítkem vyvolá spouštění funkce `mouse_drag` , která tento seznam používá. Na všech těchto objektech zavolá jejich `move` metodu o příslušný počet pixelů.

2.3 Architektura aplikace

Hierarchie souborů kódu mého projektu je:
constants.py
$\longrightarrow$ *core_classes.py*,
$\longrightarrow$ *drag_and_drop.py*, *default_interactors.py*, *default_sources.py*
$\longrightarrow$ *main.py*

Součásti položené níže v tomto seznamu závisí na těch předchozích a postupně na nich staví. Popis jednotlivých částí:

constants.py: Soubor konstant využívaných ostatními částmi, nahoře jsou konstanty měnitelné, kterými je možno přizpůsobit některé charakteristiky programu (vypnout načítání demonstračních ploch při spuštění, vypnout veškeré printování do konzole, změnit font dolní lišty, změnit barvu paprsků, změnit maximální povolený počet segmentů v paprsku, než sebekontrola programu automaticky ukončí paprsek...). Dole se nachází interně používané konstanty.

core_classes.py: Obsahuje hlavní třídy projektu - jeho základ.

drag_and_drop.py: Obsahuje kód pro výběr pomocí myši a přesun objektů. Stará se taky o vyvolání updatů pravého panelu.

default_interactors.py: Definuje předem vytvořené simulační prvky

default_sources.py: Definuje předem vytvořené zdroje paprsků

main.py: Je určen ke spuštění programu, obsahuje kód pro GUI a vytváří demo plochy při otevření programu

2.4 Formátování kódu

Soukromé atributy jsou označeny podtržítkem na začátku, třeba "_end_specified"ve tříde Segment. Python soukromé atributy neskrývá, proto můžete napsat kód, který je bude měnit přímo. To se ale nedoporučuje proto, že na práci s nimi jsou dostupné třídní metody, které jsou na jejich správné nastavení nebo přečtení přímo vytvořené.

Co nejvíc argumentům funkcí, atributům tříd a jiným proměnným jsem poskytl type anotace pro jednodušší práci s nimi a rychlejší hledání chyb.

Co se týče uspořádání kódu v souborech, úplně nahoře jsou importy z jiných souborů mého projektu, pod nimi je volný řádek a pod tím jsou importy standardních Python knihoven.

Názvy proměnných následují pravidla americké angličtiny, protože tu používají názvy v Pythonu a Tkinteru. Komentáře jsou napsané britskou angličtinou, protože tu používám já. Je to maličkost, kterou uvádím hlavně pro objasnění pro ty, kdo by se pozastavili nad tím, že používám jiný pravopis v názvu funkce a jiný v komentáři hned vedle. V kódu dbám na to, aby se pravopis shodoval s názvy proměnných v Tkinteru a v komentářích dbám, aby se shodovaly s mými poznámkami a mými shrnutími commitů v Git repozitáři.

Jelikož člověk, který chce můj kód debugovat, má můj kód stejně otevřený v editoru, zapíná se debugování přímo přepsáním konstanty v kódu. V constants.py je sekce určená nastavení, zda jsou debugovací funkce aktivní. Kromě hlavního nastavení, kterým lze zapnout a vypnout veškeré debugovací printování, je funkčnost rozdělena na několik druhů podle toho, jaký systém mého projektu chce člověk zkoumat. Může je zapínat postupně, aby konzole nebyla spamována nekonečnými print statementy, které si člověk nevyžádal. Podrobněji co ovládá co:

- debug_any: Hlavní nastavení. Pokud je vypnuto, vypne všechno debugování kromě vypsání zprávy při vypnutí.
- debug_warnings: Zapíná varování do konzole o věcech, které samy o sobě nejsou kritickou chybou, ale mohly by tropit problémy jinde, jako třeba počítání s úhly mimo rozmezí 0 až 360. Takovéto chyby se můj kód snaží nacházet a upozornit na ně. Všechna varování začínají "WARN: ". Pokud něco nefunguje a jsou při tom printována varování, zdroj těchto

varování je první místo, kam by se člověk měl podívat, protože tam asi vězí chyba.

- . . . (Okomentováno v kódu vedle jejich definic)

2.5 Simulace

Vytvořil jsem systém pro ukládání a vykreslování segmentů paprsků na Tkinter canvas. Paprsek může být definován buď dvěma body, nebo jedním bodem a jedním úhlem, což se hodí pro segmenty, u kterých není znám koncový bod, protože ještě neprošly kolizí nebo mají jít až do nekonečna a tedy být vykresleny až k okraji plochy.

Součástí simulace je také systém pro vyřizování kolizí a postupnou generaci nových segmentů, jak jde paprsek dál a podstupuje kolize, při kterých se generují nové jeho segmenty.

Kód pro kolize pochází ode mě za pomoci zdrojů o 2D analytické geometrii. Toto bylo největší úskalí projektu a rozhodnutí tvořit vlastní systém simulace byla chyba, která stála tolik úsilí, které chybělo na části projektu, které jsou aspoň vidět a na které jsem se při plánování těšil.

2.6 Popis důležitých částí kódu

Zde důležité útržky kódu tvořícího GUI. kód se nachází v main.py a je spuštěn při startu programu.

```
    # GUI setup
    root = tk.Tk()
    root.title("Nastroj na paprskovou optiku")
    # Menu bar (upper, thin)
    menubar = tk.Menu(root)
    root.configure(menu=menubar)
    # Tool bar (lower, with custom buttons)
    frm_toolbar = tk.Frame(root)
    frm_toolbar.grid(row=0, column=0, sticky="ew")
    if constants.debug_background_colors:
        frm_toolbar.configure(bg="green")

    # Selection mode selection
```

```python
    lbl_selection_label = tk.Label(master=frm_toolbar, text="Rezim vyberu: ")
    var_selection_mode = tk.StringVar()
    cbb_selection_mode = ttk.Combobox(master=frm_toolbar, values=["Vrchni objekt", "Vsechny pod kurzorem", "Segmenty car"], state="readonly", textvariable=var_selection_mode)
    cbb_selection_mode.grid(row=0, column=1, padx=(3, 0), pady=3)
    cbb_selection_mode.bind("<<ComboboxSelected>>", lambda event_: choose_selection_mode(event_, cbb_selection_mode.current()))
    cbb_selection_mode.current(0) # Set the default value as the first in the list

    pnw_panes = ttk.Panedwindow(master=root, orient=tk.HORIZONTAL)
    pnw_panes.grid(sticky="nsew", row=1, column=0)

    # Left pane
    tvw_objects = ttk.Treeview()
    pnw_panes.add(tvw_objects, weight=0)

    # Central pane
    frm_screens = tk.Frame() # Obsahuje ntb_screens a lbl_debug a grp_grip
    if constants.debug_background_colors:
        frm_screens.configure(bg="red")
    frm_screens.columnconfigure(0, weight=1)
    frm_screens.rowconfigure(0, weight=1)
    pnw_panes.add(frm_screens, weight=1)

    ntb_Screens = ttk.Notebook(master=frm_screens)
    ntb_Screens.grid(columnspan=2, sticky="nsew")

    lbl_debug = tk.Label(master=frm_screens)
    lbl_debug.grid(sticky="ew")

    grp_grip_center = ttk.Sizegrip(master=frm_screens)
    grp_grip_center.grid(column=1, row=1, sticky="ns")

    # Right pane
    frm_editing = tk.Frame()
```

```
    if constants.debug_background_colors:
        frm_editing.configure(bg="yellow")
    frm_editing.rowconfigure(0, weight=1)
    frm_editing.columnconfigure(0, weight=1)
    pnw_panes.add(frm_editing, weight=0)

    lfr_editing = tk.LabelFrame(master=frm_editing, text=
        "Detaily objektu")
    lfr_editing.grid(sticky="nsew", padx=(0, 5))

    lbl_editing_type = tk.Label(master=lfr_editing, width
        =30, text="Typ objektu: (nevybrano)")
    lbl_editing_type.grid(pady=(10, 0))
    lbl_editing_name = tk.Label(maste
    lbl_editing_name.grid(pady=(10, 0))
    sep_editing_pane = ttk.Separator(master=lfr_editing)
    sep_editing_pane.grid()
    grp_grip_right = ttk.Sizegrip(master=frm_editing)
    grp_grip_right.grid(column=0, row=1, sticky="se")
```

Listing 2.1: Widgety GUI

Zde ponechány jen instanciace widgetů, skoro všechen ostatní kód mezi jsem vymazal, protože by to bylo zbytečné vyplňování stránky. Je tady vidět instanciace všech použitých tk a ttk objektů a také část práce s bindováním událostí, jako i rovněž konfigurace některých widgetů, třeba použití ttk.ComboBox widgetu ve stavu 'readonly'.

Níže jsou popsány třídy tvořící kostru mého programu. Nacházejí se v core_classes.py a pokud chcete měnit něco v jádru programu, bude se vám hodit tento přehled.

```
class Segment:
    def __init__(self, start_x: int, start_y: int, angle:
        float, end_specified: bool =False, end_x: int =
        None, end_y: int =None, visible: bool =True):
        self.start_x: int = start_x
        self.start_y: int = start_y
        self.angle: float = angle
        self._end_specified: bool = end_specified
        self._end_x: int | None = end_x
        self._end_y: int | None = end_y
        self.visible: bool = visible
```

```
        self.last_collided_interactor = None
        self.last_collided_line = None
    def __str__(self):
        return f"Segment with X: {self.start_x} Y: {self.start_y} Angle: {self.angle}"
    def specify_end(self, x, y) -> None:
        self._end_x = x
        self._end_y = y
        self._end_specified = True
```

Listing 2.2: třída Segment

Segment popisuje jeden segment paprsku. Je definován buď startovním bodem a úhlem, nebo startovním bodem a koncovým bodem. Moje renderovací funkce se poradí s oběma způsoby definování. Často je segment vytvořen s informací o počátečním bodu a úhlu a později mu je určen koncový bod. Pro tento přechod lze použít funkci specify_end(x, y), která nastaví správně proměnné. Je doporučeno používat tuto funkci, protože je kratší, než všechny tři proměnné nastavovat manuálně a také to zamezí případům, kde by člověk zapomněl nastavit proměnnou _end_specified. Proto jsou proměnné end_x, end_y a end_specified označeny jako soukromé atributy. Funkce __str__() je dostupná pro pohodlnější debugování.

```
class Source:
    def __init__(self, x: int, y: int, angle: float) -> None:
        self.x: int = x
        self.y: int = y
        self.angle: float = angle
        self.generated_segments: list[Segment] = []
    def __str__(self) -> str:
        return f"Source with X: {self.x} Y: {self.y} Angle: {self.angle} Number of segments: {len(self.generated_segments)}"
    def generate_segments(self, parent_screen: Screen) -> None:
        ...
```

Listing 2.3: třída Source

Zdroj paprsku je definován jen svými souřadnicemi x a y a svým úhlem. Při spuštění funkce Source.generate_segments vygeneruje zdroj první paprsek s

právě těmito souřadnicemi a úhlem a další segmenty k paprsku přidává podle výsledků kolizí. Aby zdroj paprsku mohl získat data o kolizích, potřebuje dostat při zavolání této funkce odkaz na instanci plochy, na které se nacházejí prvky, se kterými bude interagovat. Z instance plochy získá seznam simulačních prvků, se kterými postupně se všemi nechá pomocí funkcí z modulu collisions.py vypočítat kolize. Všechny kolize, které neuspěly, jsou odhozeny. Z kolizí úspěšných vybere tu nejbližší k začátku posledního svého segmentu, tedy tu, na kterou paprsek narazí jako první. Segmenty vygenerované touto kolizí přidá ke svému paprsku, tím, že je přidá do svého seznamu `generated \_segments`. Opět je dostupná __str__() funkce, která vrací sesumírované podrobnosti o instanci.

```
    def __init__(self, neccessary_references: dict, canvas_width: int =1200, canvas_height: int =400):
        # Simulation-related
        self.ray_sources: list[Source] = []
        self.ray_interactors: list[Interactor] = []

        # UI-related declaration
        self.tk_frame = tk.Frame()
        self.canvas_width: int = canvas_width
        self.canvas_height: int = canvas_height
        self.tk_canvas = tk.Canvas(master=self.tk_frame, bg="#DDDDDD", width=self.canvas_width, height=self.canvas_height)
        self.tk_canvas.grid(sticky="nsew")

        # UI-related configuration
        self.tk_frame.rowconfigure(0, weight=1)
        self.tk_frame.columnconfigure(0, weight=1)
        if constants.debug_background_colors:
            self.tk_frame.configure(bg="blue")

        # Linking internal objects to canvas objects
        self.ID_to_interactor_dict: dict = {}
        ...
        # Binding
        self.tk_canvas.bind("<Motion>", lambda event: update_debug_label(event, self), add="+")
        self.tk_canvas.bind("<1>", lambda event: on_mouse_grab(event, self), add="+")
        self.tk_canvas.bind("<B1-Motion>", lambda event: on_mouse_drag(event, self), add="+")
```

```
        self.tk_canvas.bind("<B1-Motion>", lambda event: update_debug_label(event, self), add="+")
    def get_all_interactors(self) -> list[Interactor]:
        return self.ray_interactors
    def solve_all_sources(self) -> None:
        for source in self.ray_sources:
            source.generate_segments(self)
    def plot_all_interactors(self) -> None:
        # Call the plot_self() function of each object. Those also take care of deleting the old object off that canvas
        for object in self.ray_interactors:
            object.plot_self()
    def plot_all_lines(self) -> None:
        for canvas_object in self.tk_canvas.find_withtag("line"):
            self.tk_canvas.delete(canvas_object)

        for source in self.ray_sources:
            render_segments(self.tk_canvas, source.generated_segments)

    def plot_all(self) -> None:
        self.plot_all_interactors()
        self.plot_all_lines()
```

Listing 2.4: třída Screen

Asi nejcentrálnější třída, obsahuje v sobě seznam zdrojů a překážek. Všechny atributy a metody mají type anotace, aby tato třída dobře fungovala jako rozšiřitelné jádro simulace. Ze stejného důvodu tu jsou metody jako `get_all_interactors()`, které jsou připavené pro implementaci systému seskupování prvků, který jsem před nárazem na časové problémy plánoval také přidat, ale nyní to je funkce, na kterou je projekt připraven a která je možností do budoucna.

2.7 Debugovací funkce

Projekt obsahuje debugovací funkce, které mi pomáhaly při hledání chyb a potvrzování, že vše funguje tak, jak má, a určitě pomůžou i všem ostatním, kdo by se chtěli v mém projektu pohrabat nebo do něj něco přidávat. Klasické debugovací nástroje jako VSCode s Python debuggerem samozřejmě stále

jsou nejvíc důležitým nástrojem.

2.8 Použité technologie a nástroje

- Programovací jazyk: Python. S Pythonem mám předchozí zkušenosti a hodí se na tento typ projektu proto, že dovoluje jednoduše tvořit programy využívající OOP a je připravený na využití knihovny Tkinter, kterou chci používat.
- Knihovny:
  - Tkinter: Defaultní knihovna Pythonu, název je zkratka z "Tk Interface". Knihovna Tkinter byla vytvořena v roce 1991 a rozhraní v ní vytvořená vypadají dnes krajně zastarale. Mě se tento vzhled líbí, proto toto nepovažuji za negativum. V Tkinteru lze tvořit i moderně vypadající okna, a to obzvlášť díky knihovně "ttk", která dovoluje vytvářet vlastní styly.
  - Modul Tkinter.ttk: Modul, který je součástí knihovny Tkinter. Je to zkratka z "Themed Tk"a dovoluje používat několik nových grafických prvků a definovat a používat vlastní styly. Já z Tkinter.ttk používám prvky PanedWindow, Separator, LabelFrame, Treeview, Notebook a Sizegrip, které mnou vytvořenému grafickému rozhraní dávají dnes již časté moderní vymoženosti.
  - Math: Standardní knihovna Pythonu, která do mého programu dodává matematické funkce potřebné při výpočtech v analytické geometrii. Používám funkce `radians, sin, cos, sqrt, atan2, degrees`
- IDE: Visual Studio Code s rozšířením pro Python
- Verzovací systém: Git se vzdáleným repozitářem uloženým na Githubu
- Dokumentace: Vytvořena v LaTeXu za pomocí stránek Overleaf. Používá knihovny Graphics, Listings a Xcolor a další.

Během žádné části projektu nebyla využita AI.

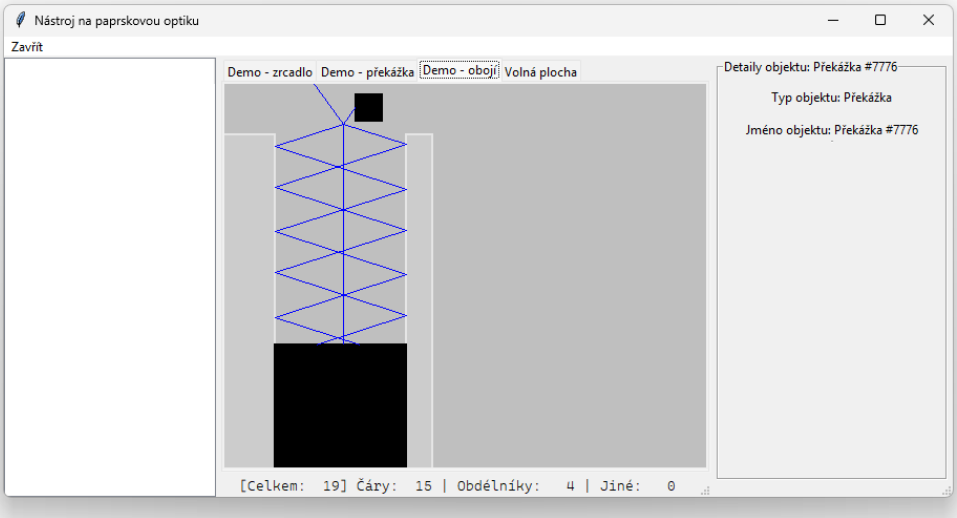
3 Uživatelská dokumentace

3.1 Instalace a spuštění

Pro používání stačí stáhnout složku obsahující soubory programu a spustit soubor main.py. Ten importuje všechny potřebné součásti a spustí okno programu. Po zavření okna se zavře zbytek programu. Projekt je závislý na aktuální verzi Pythonu, Python 3.14, a na knihovnách Math a Tkinter, obě jsou součástí Pythonu a jsou s každou jeho instalací předinstalované. Stačí tedy stáhnout Python 3.14 a program spustit.

3.2 Popis GUI

Uživatelské rozhraní je rozděleno na tři panely. Levý panel je ponechaný prázdný, zde můžou být v budoucnu přidány dodatečné ovládací prvky, nebo doimplementován widget Treeview, který tento panel zatím vyplňuje a mohl by v budoucnu podávat přehled o hierarchii objektů na jednotlivých plochách.



Obrázek 3.1: Grafické rozhraní s vykreslenou simulací a spočtenými objekty

Prostřední panel je největší a nachází se na něm hlavní pracovní plocha programu. Při škálování aplikace také dostává prioritně nový prostor, zůstává tedy největším. V horní části naleznete lištu s několika záložkami. Myší nebo klávesnicí lze mezi záložkami přepínat a tím určovat, která plocha je aktuálně zobrazena na hlavní ploše programu. Nahlédněte pod hlavní plochu a najdete lištu, na kterou se při každé změně na obrazovce vypíšou aktuální údaje o tom, kolik je na dané ploše vykresleno čar, obdélníků a jiných útvarů. Celkový počet objektů je k přečtení nalevo. Použit je font typu monospace, aby se předešlo trhání při změně počtu cifer v číslech.

Panel napravo slouží k zobrazené detailů o vybrané položce. Pokaždé, když kliknete na nějaký objekt nebo jím myší přetáhnete, je vybrán a zobrazí se zde jeho typ a jeho název. Výběr zrušíte kliknutím na volné pozadí pracovní plochy.

Nad panely se nachází lišta menu s tlačítkem "Zavřít", sem lze v budoucnu přidávat další tlačítka.

3.3 Použití

- Výběr objektu: Na objekt klikněte nebo jím myší přetáhněte
- Přemístění objektu: Na objektu podržte levé tlačítko myši a někam přesuňte, poté pusťte
- Otevření jiné plochy: Použijte lištu nad pracovní plochou. Lze použít i šipky doleva a doprava na klávesnici
- Změna velikosti okna: Rukojeti na uchopení okna se nachází v pravém dolním rohu okna a také pod pravým dolním rohem prostředního panelu
- Změna velikosti panelů: Rukojeti na pohyb hranicemi panelů najdete na předělech mezi panely
- Zavření: Použití tlačítka "Zavřít", křížku a zkratky Alt+F4 jsou ekvivalentní

3.4 Režimy výběru myší

Vlastní implementace drag 'n drop systému je důležitá součást mého projektu. Implementoval jsem dva režimy, režim lze vybrat pomocí ComboBoxu z ttk modulu. Defaultní je režim výběru jednoho objektu. Z objektů pod kurzorem vybere ten nejvíce v popředí. Režimy:

- Vrchní objekt: Z objektů přímo pod kurzorem vybere ten nejvíce v popředí
- Všechny pod kurzorem: Přetahování hýba všemi objekty pod kurzorem, k editování je vybrán ten nejvíce v popředí

3.5 Řešení problémů

Pokud se simulace na obrazovce chová jinak, než by měla, stačí obsah obrazovky aktualizovat a problém by měl hned zmizet. To lze udělat kliknutím na tlačítko "Aktualizovat"nahoře na liště nástrojů, nebo přetažením jakékoliv z překážek na obrazovce, protože to také způsobí celkovou aktualizaci obrazovky. Toto se může stát při přepínání mezi obrazovkami a tlačítko "Aktualizovat"toto řeší.

4 Závěr

GUI se vydařilo bezproblémově, jsem s ním spokojený. V aktuálním stavu funguje, jak by mělo a je zajištěno, že do budoucna na něj lze přidávat všechno, co může být potřeba.

Původně jsem zamýšlel, aby pracovní plochy byly scrollovatelné, ale to jsem v této verzi projektu nakonec neimplementoval. Zjistil jsem během práce, že by to bylo náročnější než jiné funkce, které pokládám za důležitější, zatímco přínos by nebyl tak vysoký.

Simulační část je místo, kde můj projekt pokulhává. Simulace ve svém aktuálním stavu postrádá řadu funkcí.

Rozhodl jsem se místo vytáření mnoha pokročilých funkcí vytvořit solidní základ, na kterém je možné pokročilejší simulaci vystavět.

Zjistil jsem během práce, že implementovat výpočty optiky potřebuje mnohem více času než jsem čekal, proto jsem dal prioritu vytvoření bezproblémového, fungujícího GUI.

Systém základu simulace a kód pro kolize pochází ode mě za pomoci zdrojů o 2D analytické geometrii. Toto bylo největší úskalí projektu a rozhodnutí tvořit vlastní systém simulace byla chyba, která stála tolik úsilí, které chybělo na části projektu, které jsou aspoň vidět a na které jsem se při plánování těšil. Třeba první věc, kterou chci v budoucnu implementovat je pravé editování objektů přes GUI, to je věc, co mi v mém projektu nejvíc chybí.

Cíl, který jsem si stanovil na začátku byl rozsáhlý a v průběhu tvorby jsem musel udělat kompromisy, které znamenaly, že jsem prioritizoval plně funkční a stabilní základ jakožto i GUI a neimplementoval jsem pokročilejší funkce v simulační části jako třeba systém materiálů a Snellův zákon. I přesto se práce

na simulaci stala nejvíce problematickou částí celé mé práce, kupříkladu na tvorbě kódu pro kolizi dvou čar a následném řešení problémů s touto funkcí jsem se zasekl na několik týdnů a vlastně mi to zkazilo celý dojem z práce na projektu.

Nakonec se mi i tuto část ale podařilo doladit a provést přidání aspoň nejdůležitějších funkcí, které můj projekt potřeboval. Mohl jsem posléze ve zbytku času doplnit GUI o všechny části, které jsem chtěl. Přestože v simulační části můj projekt postrádá hloubku, jsem s výsledkem svého celoročního snažení spokojený.